Linux Permissions, Processes, Logs, and Troubleshooting

Master Linux user access control, process inspection, system telemetry, log forensics, and structured operational incident troubleshooting.

DAY LEARNING OBJECTIVES

- **Objective:** Understand Linux multi-user architecture: users, groups, UIDs, GIDs, and file ownership
- **Objective:** Interpret symbolic permission strings (rwxr-xr-x) and calculate numeric octal modes (755, 644, 700)
- **Objective:** Use chmod and chown safely without resorting to dangerous anti-patterns like chmod 777
- **Objective:** Inspect running processes, process trees, and system resource consumption using ps, top, and free
- **Objective:** Examine application logs, parse error traces, and identify failure root causes with evidence
- **Objective:** Execute a structured 5-step troubleshooting methodology to diagnose and resolve service incidents safely

Topic 1: Linux Users, Groups, and File Ownership (20 mins)

Linux was architected from day one as a secure, multi-user operating system. Every single file, folder, and running process on the system is owned by a specific User and belongs to a specific Group.

User Accounts & Identifiers:
1. The Superuser (root, UID 0): Has unrestricted administrative authority over the entire operating system. It can read, write, or delete any file and terminate any process. Rule: Never run web applications or daily development as root. If a container or web app is compromised while running as root, the attacker gains full control of the host.
2. System Service Accounts (UID 1–999): Non-login service accounts created to isolate daemons (e.g., nginx, postgres, systemd-resolve).
3. Regular User Accounts (UID 1000+): Human engineers and developers (e.g. devops, priya).

Groups:
A group is a collection of user accounts sharing common access privileges (e.g. developers, sudo, docker). Instead of assigning permissions to 20 individual developers, you grant permissions to a group and add the users to that group.

Key Ownership Commands:
- whoami: Prints your current active username.
- id: Displays your User ID (UID), primary Group ID (GID), and all secondary group memberships.
- cat /etc/passwd: The system file listing all defined user accounts.
- cat /etc/group: The system file listing all groups and their members.

- **Takeaway:** Every file and process has an owner user (UID) and a primary group (GID).
- **Takeaway:** Root has UID 0 and unlimited power; never run user-facing applications as root.
- **Takeaway:** Groups allow multiple users to share access without exposing files to everyone.

Topic 2: Read, Write, and Execute Permissions (20 mins)

When you run ls -l, the very first column displays an 11-character permission string. Understanding how to decode this string is a core DevOps requirement.

``
- rwx r-x r-- 1 devops developers 4096 Oct 26 10:00 app.js
%, % % % % % % % % %
%   %   %   %
%   %   %   % % % Others (World) Permissions: r-- (Read-only)
%   %   % % % % % % % Group Permissions: r-x (Read and Execute)
%   % % % % % % % % % % % Owner (User) Permissions: rwx (Read, Write, Execute)
% % % % % % % % % % % % % File Type: (-) = Regular file, (d) = Directory, (l) = Symlink
`

What Permissions Mean for Files vs. Directories:

Permission	For a Regular File	For a Directory
r (Read)	View/read file content (cat, head)	List files inside directory (ls)
w (Write)	Modify/edit/save file content	Create, rename, or delete files inside directory
x (Execute)	Run file as a program or script	Enter directory (cd) and access file metadata

Crucial Directory Insight: To navigate into a directory (cd /var/log) or access any file inside it, the user must have execute (x) permission on that directory! If a directory has r-- without x`, you can see the file names, but you cannot enter the folder or read the contents of the files!

- **Takeaway:** Permissions are divided into three triplets: Owner (User), Group, and Others (World).
- **Takeaway:** r = Read, w = Write, x = Execute.
- **Takeaway:** On directories, x (Execute) is required to cd into the folder and access files.

Topic 3: Permission Notation: Symbolic and Octal (Numeric) Modes (25 mins)

Linux represents permissions in two interchangeable ways: Symbolic notation (rwxr-xr-x) and Octal (numeric) notation (755).

The Mathematical Octal Bitmask:
Each permission has a specific binary numeric value:
- r (Read) = 4 (binary 100)
- w (Write) = 2 (binary 010)
- x (Execute) = 1 (binary 001)
- - (None) = 0 (binary 000)

To calculate the mode for each triplet, simply add the numbers together:

``
Owner: r w x = 4 + 2 + 1 = 7
Group: r - x = 4 + 0 + 1 = 5
Other: r - - = 4 + 0 + 0 = 4
Combined Octal Mode: 754
`

Standard Industry Modes:
- 755 (rwxr-xr-x): Standard for executable scripts and directories. Owner can read/write/execute; group and others can read and execute.
- 644 (rw-r--r--): Standard for non-executable files (HTML, config files, images). Owner can read/write; group and others can only read.
- 600 (rw-------): Standard for sensitive private keys and secrets (e.g. ~/.ssh/id_rsa). Only the owner can read/write; everyone else has zero access.
- 700 (rwx------): Standard for private directories (e.g. ~/.ssh). Only the owner can enter and list files.

Why "chmod 777" is an Extreme Anti-Pattern:
When a junior engineer encounters a Permission Denied error, their first instinct is often to run:
`bash
$ chmod -R 777 /app   # DO NOT DO THIS!
`
777 (rwxrwxrwx) gives every user and process on the entire machine unrestricted write and execute permissions. Any rogue process, compromised web dependency, or malicious user can overwrite, inject malicious code into, or delete the file. Senior engineers reject pull requests containing chmod 777` immediately. Always apply the Principle of Least Privilege: grant only the minimum permissions required.

- **Takeaway:** Read = 4, Write = 2, Execute = 1. Add them together for each triplet.
- **Takeaway:** Standard file mode is 644; standard executable/dir mode is 755; SSH secret mode is 600.
- **Takeaway:** Never use chmod 777 in production; it violates the Principle of Least Privilege.

Topic 4: Basic chmod and chown Concepts (20 mins)

To modify permissions and ownership, Linux provides two primary utilities: chmod and chown.

1. chmod (Change Mode):
Modifies the access permissions of files or directories.
- Numeric Mode Syntax:
  ``bash
  $ chmod 755 deploy.sh   # rwxr-xr-x
  $ chmod 644 nginx.conf   # rw-r--r--
  $ chmod 600 id_rsa   # rw-------
  `
- Symbolic Mode Syntax:
  `bash
  # Syntax: [who][operator][permission]
  # who: u (user/owner), g (group), o (other), a (all)
  # operator: + (add), - (remove), = (set exactly)

  $ chmod u+x backup.sh   # Add execute to owner
  $ chmod g-w config.json   # Remove write from group
  $ chmod o-rwx secrets.env # Revoke all permissions from others
  `

2. chown (Change Ownership):
Transfers file ownership to a different user and/or group. Only the superuser (root) or a user with sudo privileges can change file owners.

`bash
# Change owner only:
$ sudo chown devops app.js

# Change both owner and group:
$ sudo chown devops:developers app.js

# Recursively change ownership for an entire directory tree using -R:
$ sudo chown -R www-data:www-data /var/www/html
``

- **Takeaway:** chmod changes permissions; chown changes user and group ownership.
- **Takeaway:** Use numeric chmod (755, 644) for complete mode resets; use symbolic (u+x) for targeted tweaks.
- **Takeaway:** Only root/sudo can reassign file ownership to another user.

Topic 5: Processes and Process Identification (25 mins)

A process is a program currently running in system memory. Every program executed—from a web server to a simple ls command—is assigned a unique integer called a Process ID (PID).

Process Hierarchy:
In Linux, processes form a tree hierarchy:
- The initial process started by the Linux kernel upon system boot is systemd (or init), which has PID 1.
- Every process is spawned by a Parent Process (PPID).
- If a parent process dies before its child terminates, the child process is adopted by PID 1 and can become an Orphan or a Zombie (a terminated process whose exit status has not been reaped by its parent).

Process Management Commands:
1. ps (Process Status):
  - ps aux: Displays every running process across all users:
    - a: Shows processes for all users.
    - u: Displays user-oriented format with CPU and memory percentages.
    - x: Includes processes without a controlling terminal (background daemons).
  - ps -ef | grep node: Searches for running Node.js processes.
2. kill and Signals:
  Processes communicate via numeric Signals:
  - kill -15 <PID> (SIGTERM): Graceful termination request. Gives the process time to close database connections, finish active requests, and write flush buffers. Always try SIGTERM first.
  - kill -9 <PID> (SIGKILL): Forceful immediate kill by the kernel. The process cannot catch or ignore this signal. Use only when a process is frozen or unresponsive to SIGTERM.

- **Takeaway:** Every process has a unique PID; PID 1 is the system init daemon (systemd).
- **Takeaway:** ps aux shows all running processes with user and resource usage.
- **Takeaway:** Always try graceful termination (SIGTERM / kill -15) before resorting to force kill (SIGKILL / kill -9).

Topic 6: CPU, Memory, and Disk Inspection (25 mins)

When a server slows down or an API experiences high latency, you must quickly determine whether the bottleneck is CPU saturation, Memory exhaustion, or Disk space limits.

1. top / htop (Real-Time Process Monitor):
Provides an interactive, dynamically updating dashboard of system resources:
- Load Average: Displays 3 numbers representing system load over 1, 5, and 15 minutes. On a 4-core CPU, a load average below 4.0 means the CPU has headroom; a load average of 8.0 means processes are queued waiting for CPU time.
- Press P in top to sort by CPU usage.
- Press M in top to sort by Memory usage.
- Press q to quit.

2. free -m (Memory Usage):
Inspects physical RAM and Swap usage in megabytes.
``
$ free -m
              total   used   free   shared buff/cache   available
Mem: 7960   2150   3210   120   2600   5410
Swap: 2048   0   2048
`

Crucial Metric: available RAM. Linux aggressively uses free RAM as buff/cache to speed up disk reads. This memory is automatically released the instant an application needs it. Look at the available column, not the free column, to see true usable memory!

3. df -h (Disk Free):
Displays total, used, and available disk space across all mounted filesystem partitions.
- If a partition hits 100% capacity, applications cannot write logs or temporary sessions, causing immediate service crashes.

4. uptime`:
Displays how long the server has been running since its last reboot, the number of active users, and system load averages.

- **Takeaway:** top provides real-time CPU and memory monitoring; press M to sort by memory.
- **Takeaway:** Look at available memory in free -m, not just free memory.
- **Takeaway:** df -h shows disk capacity; a 100% full disk will crash databases and loggers.

Topic 7: Application Logs and Error Messages (20 mins)

Logs are the primary source of truth when investigating what went wrong. Understanding log conventions and severity levels accelerates incident triage.

Standard Log Levels:
Applications and system services classify log events by severity:
- DEBUG: Verbose diagnostic information used during development.
- INFO: Routine operational events (e.g. Server listening on port 3000, User 45 logged in).
- WARN: Unexpected condition that does not break core functionality (e.g. API response took 800ms, Deprecated flag used).
- ERROR: A specific operation or user request failed, but the application remains running (e.g. Database query timeout, Failed to process payment).
- FATAL / CRITICAL: Catastrophic failure causing the entire service to crash or terminate (e.g. Cannot bind port 80: Address already in use, Out of Memory).

The Anatomy of an Error Trace:
A typical application crash log contains:
1. Timestamp (ISO 8601): 2026-10-26T10:14:02Z — Essential for correlating events across multiple microservices.
2. Log Level: [ERROR]
3. Component / Module: [DatabasePool]
4. Message & Reason: Connection refused at 10.0.1.4:5432
5. Stack Trace: The sequence of function calls leading to the failure. Look at the first line of the stack trace to find the file and line number where the exception occurred.

- **Takeaway:** Log severity progresses from DEBUG -> INFO -> WARN -> ERROR -> FATAL.
- **Takeaway:** Timestamps allow correlating frontend errors with backend database timeouts.
- **Takeaway:** The top of a stack trace indicates the exact file and line number of the failure.

Topic 8: A Structured Troubleshooting Workflow (25 mins)

Amateur engineers react to outages by blindly restarting services, changing random file permissions, or rebooting servers. Senior DevOps engineers follow a disciplined, hypothesis-driven troubleshooting process.

The 5-Step Operational Triage Framework:

``
[1. Gather Evidence] % % > [2. Isolate Component] % % > [3. Formulate Hypothesis] % % > [4. Test Safely] % % > [5. Verify & Document]
`

1. Step 1: Gather Evidence:
   - What is the exact symptom? (e.g. "API returns HTTP 500 on /checkout").
   - When did it start? Was a deployment, configuration change, or database migration performed recently?
   - Inspect the logs: tail -n 100 /var/log/app.log and system status systemctl status app.
2. Step 2: Isolate the Component:
   - Is the problem in DNS, Network/Firewall, Application Process, or the Database?
   - Check if the process is alive: ps aux | grep node.
   - Check if ports are listening: netstat -tlpn or ss -tulpn.
3. Step 3: Formulate a Testable Hypothesis:
   - *"The service crashed because it does not have read permissions on /etc/app/config.json."*
   - Avoid guessing multiple unrelated causes at once.
4. Step 4: Test and Apply the Fix Safely:
   - Change ONE variable at a time.
   - If applying a permission fix: chmod 644 /etc/app/config.json.
   - Never use brute-force hacks like chmod 777 or deleting databases.
5. Step 5: Verify Resolution and Prevent Recurrence:
   - Test the endpoint: curl -I http://localhost:3000/health`.
   - Monitor error rates for 15 minutes.
   - Update the deployment pipeline or Ansible playbook so future deployments never ship with the wrong permissions.

- **Takeaway:** Never blindly restart services without first gathering log evidence.
- **Takeaway:** Isolate whether the bottleneck is network, process, permissions, or resource exhaustion.
- **Takeaway:** Change only ONE variable at a time when testing a hypothesis.

INTERVIEW REVISION VAULT (TOP 5 QUESTIONS)

Q1: Explain what happens when an application tries to start but crashes with "Error: listen EADDRINUSE: address already in use :::3000". How do you diagnose and fix this? [Target Role: DevOps / SRE Engineer | Intermediate]

Model Answer: EADDRINUSE means another process is already bound to port 3000 on the network interface. To diagnose: 1) Run sudo ss -tulpn | grep 3000 or sudo lsof -i :3000 to identify the PID holding the port. 2) Inspect the process with ps -p <PID> -u to verify whether it is a zombie instance or an intentional service. 3) If it is an orphaned or rogue instance, terminate it gracefully with kill -15 <PID>. 4) Restart the intended service and verify it binds successfully.

Key Evaluation Signals: Identify process holding port with ss -tulpn or lsof -i. • Inspect PID details before terminating. • Send SIGTERM (15) to release the socket cleanly.

Common Candidate Pitfalls: Changing the application port randomly instead of freeing the blocked port. • Rebooting the entire server.

Q2: Why is "chmod 777" considered an unacceptable practice in cloud environments, and what should you do instead when facing permission errors? [Target Role: DevOps / Cloud Security | Beginner]

Model Answer: chmod 777 grants universal read, write, and execute permissions to everyone, completely violating the Principle of Least Privilege. If a web application with 777 permissions suffers a remote code execution vulnerability, an attacker can modify binaries, inject webshells, or wipe the disk. Instead, you should determine the exact user running the service (e.g. www-data or nodeapp), ensure that specific user or their group owns the directory using chown, and grant minimal permissions like 755 for directories and 644 for files.

Key Evaluation Signals: 777 grants full write/execute to everyone, creating critical security risks. • Enforce Principle of Least Privilege. • Set ownership with chown and use 755/644 standard modes.

Common Candidate Pitfalls: Claiming 777 is fine if the server is inside a private VPC. • Not explaining the alternative (chown + 755/644).

Q3: What is the difference between a Zombie process and an Orphan process in Linux? [Target Role: Linux / Systems Administrator | Intermediate]

Model Answer: An Orphan process is a running process whose parent process terminated before it did; it is immediately adopted by PID 1 (systemd/init), which monitors it until completion. A Zombie process (state Z in ps) is a process that has completed execution and terminated, but its entry remains in the process table because its parent has not yet read its exit status via the wait() system call. Zombies do not consume CPU or RAM, but they consume process table slots. If a parent continuously leaks zombies, the system can run out of available PIDs.

Key Evaluation Signals: Orphan is still running; adopted by PID 1. • Zombie is dead; waiting for parent to read exit status. • Too many zombies exhaust the system PID space.

Common Candidate Pitfalls: Thinking you can kill a zombie with kill -9 (zombies are already dead; you must signal the parent). • Confusing orphans with background daemons.

Q4: How do you interpret the three numbers in the "load average" line of uptime or top? [Target Role: Site Reliability Engineer | Intermediate]

Model Answer: The three numbers represent the average number of processes in a runnable or uninterruptible sleep state (waiting for CPU or non-interruptible disk I/O) over the last 1 minute, 5 minutes, and 15 minutes. To interpret them, you must compare them against the total number of CPU cores available. On an 8-core CPU, a load average of 8.0 indicates 100% capacity. If the 1-minute load is 16.0 while the 15-minute load is 2.0, the system is experiencing a sudden severe traffic spike.

Key Evaluation Signals: Averages over 1, 5, and 15 minutes. • Must be evaluated relative to total number of CPU cores. • High uninterruptible load often indicates disk I/O bottlenecks rather than pure CPU.

Common Candidate Pitfalls: Interpreting load average as a percentage without knowing the core count. • Forgetting that disk I/O wait contributes to load average in Linux.

Q5: Walk me through your troubleshooting approach when a microservice in production returns HTTP 500 Internal Server Error. [Target Role: DevOps / Platform Engineer | Scenario]

Model Answer: I follow a structured triage process: 1) Verify the scope: Is the 500 error occurring for all requests or only specific routes (e.g. /checkout)? 2) Check application logs: Tail the active log stream (tail -n 100 app.log) to locate the exact unhandled exception and stack trace. 3) Check system telemetry: Run free -m and df -h to verify the server is not out of memory or disk space. 4) Check dependencies: Verify backend databases and upstream APIs are reachable and healthy. 5) Mitigate immediately: If caused by a recent bad deployment, trigger an automated rollback to the previous stable release. 6) Post-incident: Conduct a blameless postmortem to add regression tests and health checks.

Key Evaluation Signals: Determine scope and recent change history. • Inspect error logs for stack traces. • Verify system resource limits (RAM, disk) and downstream dependencies. • Prioritize rapid customer mitigation (rollback) before deep code investigation.

Common Candidate Pitfalls: Starting by editing code directly on the production server. • Failing to consider rolling back to restore customer service immediately.

DAY SUMMARY & RAPID REVISION CHECKLIST

- **Verified Concept:** Linux permissions are divided into Owner (u), Group (g), and Others (o) triplets.
- **Verified Concept:** Read = 4, Write = 2, Execute = 1. Standard file mode is 644; standard executable/dir mode is 755.
- **Verified Concept:** Execute bit (x) on a directory is required to cd into the folder and access files within it.
- **Verified Concept:** chmod changes access permissions; chown changes user and group ownership.
- **Verified Concept:** Never use chmod 777 in production; it violates the Principle of Least Privilege and exposes secrets.
- **Verified Concept:** ps aux displays running processes; always send SIGTERM (kill -15) before resorting to SIGKILL (kill -9).

- **Verified Concept:** free -m: Look at the "available" column, not free; df -h checks disk partition capacity.
- **Verified Concept:** Follow the 5-step troubleshooting framework: Gather evidence -> Isolate -> Hypothesize -> Fix -> Verify.